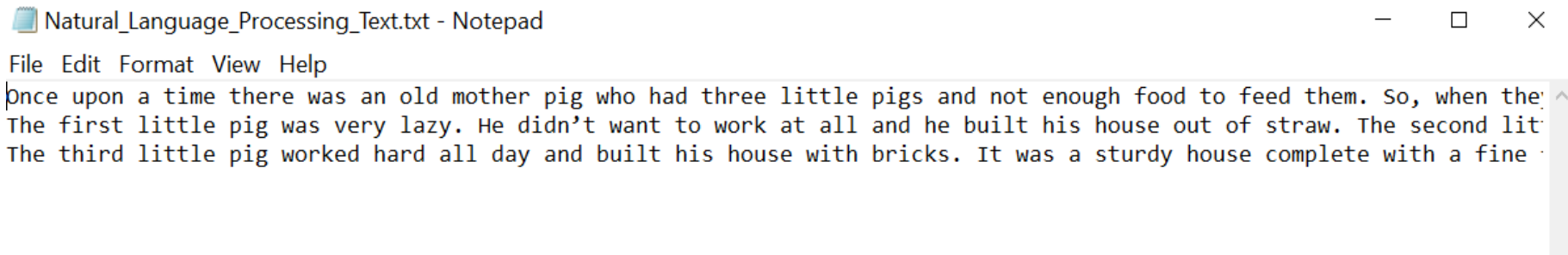


Exploring features of NLTK

Open the text file for processing:

- Consider we have a .txt file saved in the desktop.



- First, we are going to open and read the file which we want to analyze.

```
#Open the text file :  
text_file = open("Natural_Language_Processing_Text.txt")
```

```
#Read the data :  
text = text_file.read()
```

```
#Datatype of the data read :  
print (type(text))  
print("\n")
```

<class 'str'>

```
#Print the text :  
print(text)  
print("\n")  
#Length of the text :  
print (len(text))
```

Once upon a time there was an old mother pig who had three little pigs and not enough food to feed them. So when they were old enough, she sent them out into the world to seek their fortunes.

The first little pig was very lazy. He didn't want to work at all and he built his house out of straw. The second little pig worked a little bit harder but he was somewhat lazy too and he built his house out of sticks. Then, they sang and danced and played together the rest of the day.

The third little pig worked hard all day and built his house with bricks. It was a sturdy house complete with a fine fireplace and chimney. It looked like it could withstand the strongest winds.

675

- Next, notice that the data type of the text file read is a **String**. The number of characters in our text file is **675**

- If you are using colab
- Mount the drive as below:

```
✓ [15] from google.colab import drive  
drive.mount('/content/gdrive')
```

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True)

```
✓ [16] #Opening the text file  
text_file=open('/content/gdrive/MyDrive/1. Teaching Materials/GCES/NLP/Natural_Language_Processing_Text.txt')
```

Importing required libraries

- For various data processing cases in NLP, we need to import some libraries.
- In this case, we are going to use NLTK for Natural Language Processing.
- We will use it to perform various operations on the text.

```
#Import required libraries :  
import nltk  
from nltk import sent_tokenize  
from nltk import word_tokenize  
nltk.download('punkt')
```

Sentence tokenizing:

- It is a process of **breaking down a paragraph by sentences**.
- By tokenizing the text with **sent_tokenize()**, we can get the text as sentences.

```
#Tokenize the text by sentences :
sentences = sent_tokenize(text)

#How many sentences are there? :
print (len(sentences))

#Print the sentences :
#print(sentences)
sentences
```

9

```
['Once upon a time there was an old mother pig who had three little pigs and not enough food to feed them.',
 'So when they were old enough, she sent them out into the world to seek their fortunes.',
 'The first little pig was very lazy.',
 'He didn't want to work at all and he built his house out of straw.',
 'The second little pig worked a little bit harder but he was somewhat lazy too and he built his house out of sticks.',
 'Then, they sang and danced and played together the rest of the day.',
 'The third little pig worked hard all day and built his house with bricks.',
 'It was a sturdy house complete with a fine fireplace and chimney.',
 'It looked like it could withstand the strongest winds.']
```

Word tokenizing:

- Process of **breaking down paragraphs by words**.
- By tokenizing the text with **word_tokenize()**, we can get the text as words.

```
#Tokenize the text with words :  
words = word_tokenize(text)
```

```
#How many words are there? :  
print (len(words))
```

```
#Print words :  
print (words)
```

```
144  
['Once', 'upon', 'a', 'time', 'there', 'was', 'an', 'old', 'mother', 'pig', 'who', 'had', 'three', 'little', 'pigs', 'and', 'no',  
t', 'enough', 'food', 'to', 'feed', 'them', '.', 'So', 'when', 'they', 'were', 'old', 'enough', ',', 'she', 'sent', 'them', 'ou',  
t', 'into', 'the', 'world', 'to', 'seek', 'their', 'fortunes', '.', 'The', 'first', 'little', 'pig', 'was', 'very', 'lazy',  
, '.', 'He', 'did', 'not', 'want', 'to', 'work', 'at', 'all', 'and', 'he', 'built', 'his', 'house', 'out', 'of', 'straw', '.', 'T',  
he', 'second', 'little', 'pig', 'worked', 'a', 'little', 'bit', 'harder', 'but', 'he', 'was', 'somewhat', 'lazy', 'too', 'and',  
'he', 'built', 'his', 'house', 'out', 'of', 'sticks', '.', 'Then', ',', 'they', 'sang', 'and', 'danced', 'and', 'played', 'togeth',  
er', 'the', 'rest', 'of', 'the', 'day', '.', 'The', 'third', 'little', 'pig', 'worked', 'hard', 'all', 'day', 'and', 'built',  
'his', 'house', 'with', 'bricks', '.', 'It', 'was', 'a', 'sturdy', 'house', 'complete', 'with', 'a', 'fine', 'fireplace', 'an',  
d', 'chimney', '.', 'It', 'looked', 'like', 'it', 'could', 'withstand', 'the', 'strongest', 'winds', '.']
```

- Next, we can see the entire text of our data is represented as words and also notice that the total number of words here is **144**.

Find the frequency distribution:

```
#Import required libraries :  
from nltk.probability import FreqDist  
  
#Find the frequency :  
fdist = FreqDist(words)  
  
#Print 10 most common words :  
fdist.most_common(10)
```

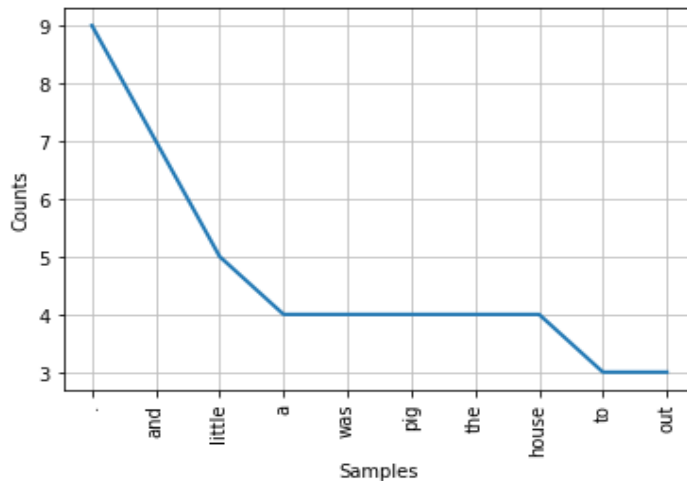
```
[('.', 9),  
 ('and', 7),  
 ('little', 5),  
 ('a', 4),  
 ('was', 4),  
 ('pig', 4),  
 ('the', 4),  
 ('house', 4),  
 ('to', 3),  
 ('out', 3)]
```

- Notice that the **most used words** are **punctuation marks** and **stopwords**.
- We will have to **remove such words** to analyze the actual text.

Plot the frequency graph:

- Let's plot a graph to visualize the word distribution in our text.

```
#Plot the graph for fdist :  
import matplotlib.pyplot as plt  
  
fdist.plot(10)
```



- In the graph above, notice that a **period “.”** is used **nine times** in our text.
- Analytically speaking, **punctuation marks are not that important** for natural language processing.
- Therefore, in the next step, we will be **removing such punctuation marks**.

Remove punctuation marks:

- we are going to remove the **punctuation** marks as they are **not very useful** for us.
- We are going to use **isalpha()** method to separate the punctuation marks from the actual text.
- Also, we are going to make a new list called **words_no_punc**, which will store the words in lower case but exclude the punctuation marks.

```
#Empty list to store words:
words_no_punc = []

#Removing punctuation marks :
for w in words:
    if w.isalpha():
        words_no_punc.append(w.lower())

#Print the words without punctuation marks :
print (words_no_punc)

print ("\n")

#Length :
print (len(words_no_punc))
```

```
['once', 'upon', 'a', 'time', 'there', 'was', 'an', 'old', 'mother', 'pig', 'who', 'had', 'three', 'little', 'pigs', 'and', 'no', 't', 'enough', 'food', 'to', 'feed', 'them', 'so', 'when', 'they', 'were', 'old', 'enough', 'she', 'sent', 'them', 'out', 'int', 'o', 'the', 'world', 'to', 'seek', 'their', 'fortunes', 'the', 'first', 'little', 'pig', 'was', 'very', 'lazy', 'he', 'did', 'wa', 'nt', 'to', 'work', 'at', 'all', 'and', 'he', 'built', 'his', 'house', 'out', 'of', 'straw', 'the', 'second', 'little', 'pig', 'worked', 'a', 'little', 'bit', 'harder', 'but', 'he', 'was', 'somewhat', 'lazy', 'too', 'and', 'he', 'built', 'his', 'house', 'out', 'of', 'sticks', 'then', 'they', 'sang', 'and', 'danced', 'and', 'played', 'together', 'the', 'rest', 'of', 'the', 'day', 'the', 'third', 'little', 'pig', 'worked', 'hard', 'all', 'day', 'and', 'built', 'his', 'house', 'with', 'bricks', 'it', 'was', 'a', 'sturdy', 'house', 'complete', 'with', 'a', 'fine', 'fireplace', 'and', 'chimney', 'it', 'looked', 'like', 'it', 'could', 'withstand', 'the', 'strongest', 'winds']
```

132

- As shown above, all the punctuation marks from our text are excluded.
- These can also be cross-checked with the number of words.

Displaying words without punctuation

- Printing the **ten most common words** from the sample text.

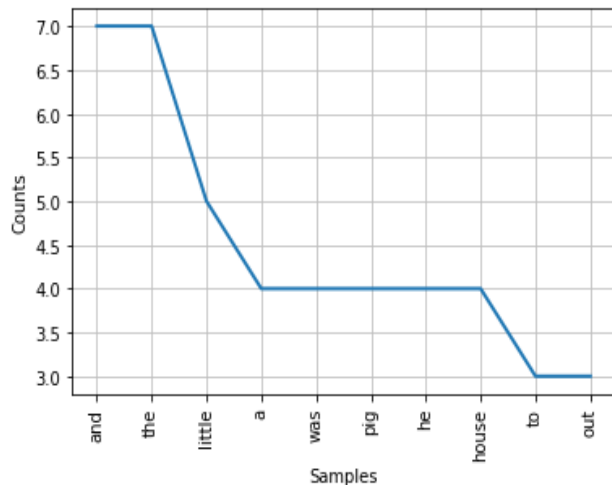
```
#Frequency distribution :  
fdist = FreqDist(words_no_punc)  
  
fdist.most_common(10)
```

```
[('and', 7),  
 ('the', 7),  
 ('little', 5),  
 ('a', 4),  
 ('was', 4),  
 ('pig', 4),  
 ('he', 4),  
 ('house', 4),  
 ('to', 3),  
 ('out', 3)]
```

Plotting graph without punctuation marks:

```
#Plot the most common words on graph:
```

```
fdist.plot(10|
```



- Notice that **we still have many words** that **are not very useful** in the analysis of our text file sample, such as **“and,” “but,” “so,”** and others.
- Next, we need to **remove coordinating conjunctions**.

Importing stopwords from NLTK

```
nltk.download('stopwords')
```

```
from nltk.corpus import stopwords  
  
#List of stopwords  
stopwords = stopwords.words("english")  
print(stopwords)
```

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'y  
ourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it's", 'its', 'itself',  
'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those',  
'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'a  
n', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'b  
etween', 'into', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'of  
f', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both',  
'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very',  
's', 't', 'can', 'will', 'just', 'don', "don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'ar  
en', "aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven', "have  
n't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "should  
n't", 'wasn', "wasn't", 'weren', "weren't", 'won', "won't", 'wouldn', "wouldn't"]
```

Removing stopwords:

```
#Empty list to store clean words :
clean_words = []

for w in words_no_punc:
    if w not in stopwords:
        clean_words.append(w)

print(clean_words)
print("\n")
print(len(clean_words))
```

```
['upon', 'time', 'old', 'mother', 'pig', 'three', 'little', 'pigs', 'enough', 'food', 'feed', 'old', 'enough', 'sent', 'world',
'seek', 'fortunes', 'first', 'little', 'pig', 'lazy', 'want', 'work', 'built', 'house', 'straw', 'second', 'little', 'pig', 'wo
rked', 'little', 'bit', 'harder', 'somewhat', 'lazy', 'built', 'house', 'sticks', 'sang', 'danced', 'played', 'together', 'res
t', 'day', 'third', 'little', 'pig', 'worked', 'hard', 'day', 'built', 'house', 'bricks', 'sturdy', 'house', 'complete', 'fin
e', 'fireplace', 'chimney', 'looked', 'like', 'could', 'withstand', 'strongest', 'winds']
```

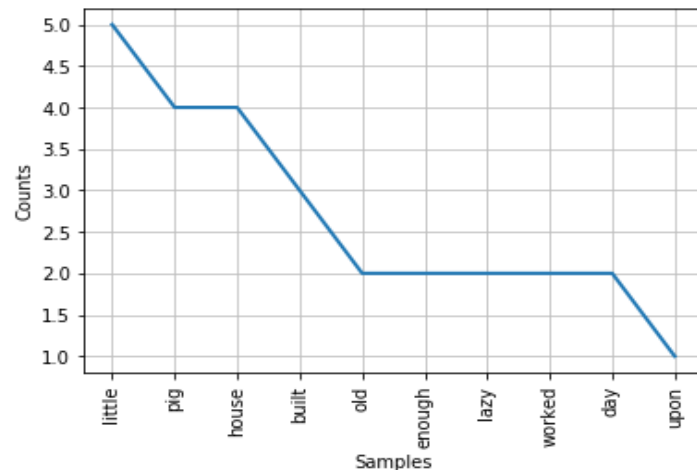
65

Final frequency distribution:

```
#Frequency distribution :  
fdist = FreqDist(clean_words)  
  
fdist.most_common(10)
```

```
[('little', 5),  
 ('pig', 4),  
 ('house', 4),  
 ('built', 3),  
 ('old', 2),  
 ('enough', 2),  
 ('lazy', 2),  
 ('worked', 2),  
 ('day', 2),  
 ('upon', 1)]
```

```
#Plot the most common words on graph:  
  
fdist.plot(10)
```



- As shown above, the final graph has many useful words that help us understand what our sample data is about, showing **how essential it is to perform data cleaning on NLP**.
- Next, we will cover various topics in NLP with coding examples.

Word Cloud

- Word Cloud is a data visualization technique in which words from a given text are displayed on the main chart.
- In this technique, more frequent or essential words are displayed in a larger and bolder font, while less frequent or essential words display in smaller or thinner fonts.
- It is a beneficial technique in NLP that gives us a glance at what text should be analyzed.

Assignment:

1. What do you mean by data preprocessing?
2. What are the techniques for data preprocessing in NLP?
3. Why is it necessary to preprocess data in NLP?
4. What do you mean by stop words? Why are they needed to be removed from our dataset?

Word Cloud Python Implementation:

```
#Library to form wordcloud :  
from wordcloud import WordCloud  
  
#Library to plot the wordcloud :  
import matplotlib.pyplot as plt  
  
#Generating the wordcloud :  
wordcloud = WordCloud().generate(text)  
  
#Plot the wordcloud :  
plt.figure(figsize = (12, 12))  
plt.imshow(wordcloud)  
  
#To remove the axis value :  
plt.axis("off")  
plt.show()
```


Stemming:

- We use **Stemming** to **normalize words**.
- In English and many other languages, a single word can take multiple forms depending upon context used.
- For instance, the verb “study” can take many forms like “studies,” “studying,” “studied,” and others, depending on its context.
- When we tokenize words, an interpreter considers these input words as different words even though their underlying meaning is the same.
- Moreover, as we know that NLP is about analyzing the meaning of content, to resolve this problem, we use stemming.
- Stemming normalizes the word by truncating the word to its stem word.
- For example, the words “studies,” “studied,” “studying” will be reduced to “**studi**,” making all these word forms to refer to only one token.
- Notice that **stemming may not give us a dictionary**, grammatical word for a particular set of words.
- Popular stemmer are **PorterStemmer** and **SnowballStemmer**. There are others also.

Porter's Stemmer Example 1:

```
#Stemming Example :  
  
#Import stemming library :  
from nltk.stem import PorterStemmer  
  
porter = PorterStemmer()  
  
#Word-list for stemming :  
word_list = ["Study", "Studying", "Studies", "Studied"]  
  
for w in word_list:  
    print(porter.stem(w))  
  
studi  
studi  
studi  
studi
```

- In the code snippet below, we show that all the words truncate to their stem words. However, notice that the stemmed word is not a dictionary word.

Porter's Stemmer Example 2:

```
#Stemming Example :  
  
#Import stemming library :  
from nltk.stem import PorterStemmer  
  
porter = PorterStemmer()  
  
#Word-list for stemming :  
word_list = ["studies", "leaves", "decreases", "plays"]  
  
for w in word_list:  
    print(porter.stem(w))
```

```
studi  
leav  
decreas  
play
```

SnowballStemmer:

- SnowballStemmer generates the **same output as porter stemmer**, but it **supports many more languages**.

```
: #Stemming Example :  
  
#Import stemming library :  
from nltk.stem import SnowballStemmer  
  
snowball = SnowballStemmer("english")  
  
#Word-List for stemming :  
word_list = ["Study", "Studying", "Studies", "Studied"]  
  
for w in word_list:  
    print(snowball.stem(w))
```

```
studi  
studi  
studi  
studi
```

Languages supported by snowball stemmer:

```
#Stemming Example :
```

```
#Import stemming library :
```

```
from nltk.stem import SnowballStemmer
```

```
#Print languages supported :
```

```
SnowballStemmer.languages
```

```
('arabic',  
'danish',  
'dutch',  
'english',  
'finnish',  
'french',  
'german',  
'hungarian',  
'italian',  
'norwegian',  
'porter',  
'portuguese',  
'romanian',  
'russian',  
'spanish',  
'swedish')
```

Lemmatization:

- Lemmatization tries to achieve a similar base “stem” for a word.
- However, what makes it different is that it finds the dictionary word instead of truncating the original word.
- Stemming does not consider the context of the word. That is why it generates results faster, but it is less accurate than lemmatization.
- If accuracy is not the project’s final goal, then stemming is an appropriate approach.

- Lemmatization **takes into account Part Of Speech (POS) values.**
- Also, lemmatization may generate different outputs for different values of POS.
- We generally have four choices for POS:

Verb (v)
Examples : Study, Play, Learn, Am, Is, Are...
Noun (n)
Examples : Doctor, Engineer, Farm, Physiotherapist, Towards AI...
Adjective (a)
Examples : Beautiful, Elegant, Angry, Polite, Repulsive...
Adverb (r)
Examples : Badly, Slowly, Peacefully, Very, Extremely, Occasionally...

- In the following example, we are taking the PoS tag as “**verb**,” and when we apply the lemmatization rules, it **gives us dictionary words** instead of truncating the original word:

```
nltk.download('wordnet')
```

```
from nltk import WordNetLemmatizer

lemma = WordNetLemmatizer()
word_list = ["Study", "Studying", "Studies", "Studied"]

for w in word_list:
    print(lemma.lemmatize(w ,pos="v"))
```

```
Study
Studying
Studies
Studied
```

Lemmatizer with default PoS value

- The **default value of PoS in lemmatization is a noun(n)**. In the following example, we can see that it's generating dictionary words:

```
from nltk import WordNetLemmatizer

lemma = WordNetLemmatizer()
word_list = ["studies", "leaves", "decreases", "plays"]

for w in word_list:
    print(lemma.lemmatize(w))
```

```
study
leaf
decrease
play
```

```
from nltk import WordNetLemmatizer

lemma = WordNetLemmatizer()
word_list = ["am", "is", "are", "was", "were"]

for w in word_list:
    print(lemma.lemmatize(w, pos="v"))
```

be
be
be
be
be

Lemmatizer with different POS values

```
from nltk.stem import WordNetLemmatizer  
  
lemmatizer = WordNetLemmatizer()  
print(lemmatizer.lemmatize('studying', pos="v"))  
print(lemmatizer.lemmatize('studying', pos="n"))  
print(lemmatizer.lemmatize('studying', pos="a"))  
print(lemmatizer.lemmatize('studying', pos="r"))
```

study
studying
studying
studying

Stemmer vs Lemmatizer:

- Stemming and lemmatization are techniques used to reduce words to their root forms. Stemming involves removing the suffixes from words, such as "ing" or "ed," to reduce them to their base form. For example, the word "jumping" would be stemmed to "jump."
- Lemmatization, however, involves reducing words to their base form based on their part of speech. For example, the word "jumped" would be lemmatized to "jump," but the word "jumping" would be lemmatized to "jumping" since it is a present participle.

```
from nltk.stem import PorterStemmer  
  
stemmer = PorterStemmer()  
  
print(stemmer.stem('studies'))
```

studi

```
from nltk.stem import WordNetLemmatizer  
  
lemmatizer = WordNetLemmatizer()  
  
print(lemmatizer.lemmatize('studies'))
```

study

- on stemming, the word “studies” gets truncated to “studi.”
- During lemmatization, the word “studies” displays its dictionary word “study.”

Assignment:

1. Define a stemmer. What is the role of stemmer in NLP?
2. Explain word stemming with relevant example.
3. Define a lemmatizer. What is the role of lemmatizer in NLP?
4. Explain lemmatization with relevant example.
5. Differentiate between stemming and lemmatization.

Part of Speech Tagging

Can you help me with the can?

Can you help me with the can?

- Parts of speech(PoS) tagging is crucial for **syntactic** and **semantic analysis**.
- Therefore, for something like the sentence above, the word “can” has several semantic meanings.
- The first “can” is used for question formation.
- The second “can” at the end of the sentence is used to represent a container.
- The first “can” is a verb, and the second “can” is a noun.
- Giving the word a specific meaning allows the program to handle it correctly in both semantic and syntactic analysis.

A simple example demonstrating PoS tagging.

```
#PoS tagging :  
tag = nltk.pos_tag(["Studying","Study"])  
print (tag)
```

```
[('Studying', 'VBG'), ('Study', 'NN')]
```

A simple example demonstrating PoS tagging.

#PoS tagging example :

```
sentence = "A very beautiful young lady is walking on the beach"
```

#Tokenizing words :

```
tokenized_words = word_tokenize(sentence)
```

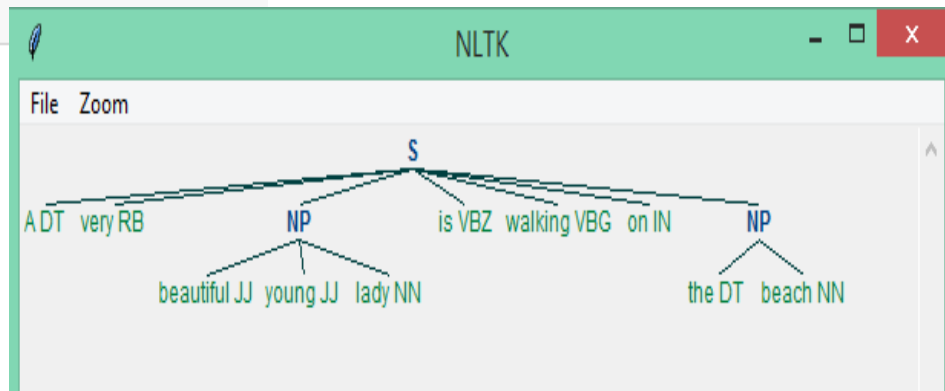
```
for words in tokenized_words:
```

```
    tagged_words = nltk.pos_tag(tokenized_words)
```

```
tagged_words
```

```
[('A', 'DT'),  
 ('very', 'RB'),  
 ('beautiful', 'JJ'),  
 ('young', 'JJ'),  
 ('lady', 'NN'),  
 ('is', 'VBZ'),  
 ('walking', 'VBG'),  
 ('on', 'IN'),  
 ('the', 'DT'),  
 ('beach', 'NN')]
```

- In this example, we can see that we have successfully extracted the **noun phrase** from the text.



List of PoS tags

- | | | | |
|-----|---|-----|----------------------------|
| 1. | CC: Coordinating Conjunction | 11. | MD: Modal |
| 2. | CD: Cardinal Digit | 12. | NN: Noun, Singular |
| 3. | DT: Determiner | 13. | NNS: Noun, Plural |
| 4. | EX: Existential There | 14. | NNP: Proper Noun, Singular |
| 5. | FW: Foreign Word | 15. | NNPS: Proper Noun, Plural |
| 6. | IN: Preposition / Subordinating Conjunction | 16. | PDT: Predeterminer |
| 7. | JJ: Adjective | 17. | POS: Possessive Endings |
| 8. | JJR: Adjective, Comparative | 18. | PRP: Personal Pronoun |
| 9. | JJS: Adjective, Superlative | 19. | PRP\$: Possessive Pronoun |
| 10. | LS: List Marker | 20. | RB: Adverb |

List of PoS tags

- | | | | |
|-----|---|-----|----------------------------|
| 1. | CC: Coordinating Conjunction | 11. | MD: Modal |
| 2. | CD: Cardinal Digit | 12. | NN: Noun, Singular |
| 3. | DT: Determiner | 13. | NNS: Noun, Plural |
| 4. | EX: Existential There | 14. | NNP: Proper Noun, Singular |
| 5. | FW: Foreign Word | 15. | NNPS: Proper Noun, Plural |
| 6. | IN: Preposition / Subordinating Conjunction | 16. | PDT: Predeterminer |
| 7. | JJ: Adjective | 17. | POS: Possessive Endings |
| 8. | JJR: Adjective, Comparative | 18. | PRP: Personal Pronoun |
| 9. | JJS: Adjective, Superlative | 19. | PRP\$: Possessive Pronoun |
| 10. | LS: List Marker | 20. | RB: Adverb |

- | | |
|--|--|
| 21. RBR: Adverb, Comparative | 31. VBZ: Verb, Present Tense, Third Person Singular |
| 22. RBS: Adverb, Superlative | 32. WDT: Wh — Determiner |
| 23. RP: Particle | 33. WP: Wh — Pronoun |
| 24. TO: To | 34. WP\$: Possessive Wh — Pronoun |
| 25. UH: Interjection | 35. WRB: Wh — Adverb |
| 26. VB: Verb, Base Form | |
| 27. VBD: Verb, Past Tense | |
| 28. VBG: Verb, Present Participle | |
| 29. VBN: Verb, Past Participle | |
| 30. VBP: Verb, Present Tense, Not Third Person Singular | |

- | | |
|--|--|
| 21. RBR: Adverb, Comparative | 31. VBZ: Verb, Present Tense, Third Person Singular |
| 22. RBS: Adverb, Superlative | 32. WDT: Wh — Determiner |
| 23. RP: Particle | 33. WP: Wh — Pronoun |
| 24. TO: To | 34. WP\$: Possessive Wh — Pronoun |
| 25. UH: Interjection | 35. WRB: Wh — Adverb |
| 26. VB: Verb, Base Form | |
| 27. VBD: Verb, Past Tense | |
| 28. VBG: Verb, Present Participle | |
| 29. VBN: Verb, Past Participle | |
| 30. VBP: Verb, Present Tense, Not Third Person Singular | |

(CC) Coordinating Conjunction

Words

And, But, For, Nor, Or, So, Yet...

Example

You can eat your cake with a spoon **or** fork.

(CD) Cardinal Digit

Words

One, Two, Three, Four, Five, Six...

Example

There are **five** flowers in the flower vase.

(EX) Existential There

Words

There

Example

There is a pen **on** the desk.

(FW) Foreign Words

Words

dolce, ersatz, esprit, quo, maitre

Example

They were selling **ersatz** products.

(DT) Determiner

Definite Article

The

Indefinite Article

A, An

Demonstratives

This, That, These, Those

Pronouns and Possessive Determiners

My, Your, His, Her, It's, Our, Their

Quantifiers

Any, A few, A little, Much, Many, Most, Some

Numbers

One, Ten, Thirty

Distributives

All, Both, Half, Either, Neither, Each, Every

Difference Words

Other, Another

Pre-determiners

Such, What, Rather, Quite

(IN) Preposition / Subordinating Conjunction

Words

At, In, On, Above, Behind

Example

All speak **at** the same time.

(JJ) Adjective

Words

New, Old, High, Special, Big, Local

Example

They live in a beautiful house.

(JJR) Adjective, Comparative

Words

Newer, Bigger, Higher

Example

The new bird is angrier than Robin.

(JJR) Adjective, Superlative

Words

Biggest, Darkest, Most

Example

Jupyter is the biggest planet in our solar system.

(MD) Modal

Words

Could, Will, Should

Example

Should we go outside in this situation?

(LS) List Marker

Words

1), 2), 3)...

Example

1) One
2) Two
3) Three

(NN) Noun , Singular

Words

Year, Home, Cost, Time, Education

Example

Why don't we stop by my house for coffee?

(NNS) Noun , Plural

Words

Pens, Books, Hats

Example

There are a bunch of pens on the table.

(NNP) Proper Noun , Singular

Words

April, Africa, Pratik

Example

April is one of the hottest months.

(NNPS) Proper Noun , Plural

Words

Americans, Indians

Example

Most Americans are young at heart.

(PDT) Predeterminer

Words

All, Both, Half

Example

They all were accepted.

(POS) Possessive Endings

Words

Parent's, Pratik's

Example

It's Jan's assignment.

(PRP) Personal Pronoun

Words

I, He, She

Example

She is very brilliant.

(PRP\$) Possessive Pronoun

Words

My, His, Her, Hers, Yours

Example

This is my book.

(RB) Adverb

Words

Really, Already, Still, Early, Now

Example

We are **still** waiting for them.

(RBR) Adverb, Comparative

Words

Worse, Less, Better

Example

This is way **better** than we anticipated.

(RBS) Adverb, Superlative

Words

Worst, Least, Best

Example

This is the **best** day ever.

(RP) Adverb, Superlative

Words

Aboard, About, Across, Along, At, Away

Example

They were **about** to leave.

(TO) To

Words

To

Example

We were going **to** the store.

(UH) Interjection

Words

Amen, Huh, Howdy, Dammit, Heck, Anyways

Example

What the **heck**!

(VB) Verb, Base Form

Words

Take, Play, Sit, Listen

Example

Let us **play** a game.

(VBN) Verb, Past Participle

Words

Taken, Languished, Dilapidated

Example

They were **taken** hostages.

(VBD) Verb, Past Tense

Words

Took, Studied, Played

Example

We **studied** for an hour.

(VBP) Verb, Present Tense, Not 3rd Person

Words

Terminate, Appear, Cure

Example

We are going to **terminate** the contract.

(VBG) Verb, Present Participle

Words

Playing, Studying

Example

They have been **playing** since morning.

(VBP) Verb, Present Tense, 3rd Person

Words

Uses, Speaks, Slaps

Example

He generally **speaks** about sports.

(WP) Wh - Pronoun
Words
Who, Whom, Which, What
Example
Who is that guy in the limo?

(WP\$) Possessive Wh - Pronoun
Words
Whose
Example
Whose pen is this?

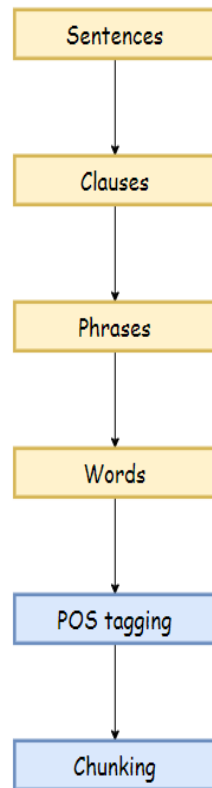
(WRB) Wh - Adverb
Words
How, However, Why, Where, When
Example
How is this possible?

Chunking:

- Chunking means to **extract meaningful phrases** from unstructured text.
- **By tokenizing** a book into words, it's sometimes **hard to infer meaningful information**.
- It works on top of Part of Speech(PoS) tagging.
- Chunking **takes PoS tags** as input and **provides chunks** as output.
- Chunking literally means a group of words, which **breaks simple text into phrases** that are **more meaningful** than individual words.

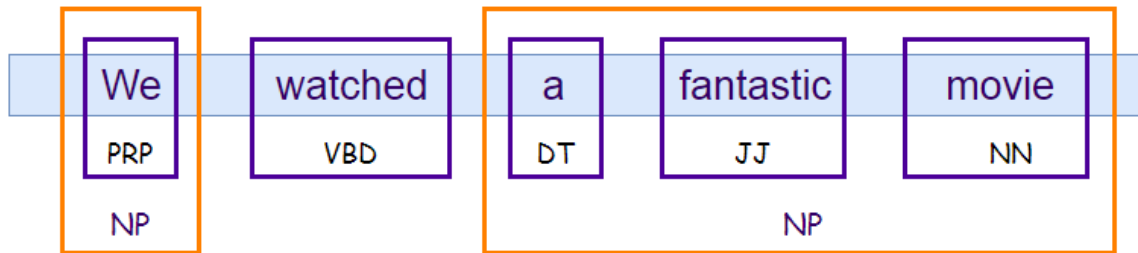
The chunking process in NLP

- Before working with an example, we need to know what phrases are?
- **Meaningful groups of words are called phrases.**
- There are five significant categories of phrases.
 - Noun Phrases (NP).
 - Verb Phrases (VP).
 - Adjective Phrases (ADJP).
 - Adverb Phrases (ADVP).
 - Prepositional Phrases (PP).



Phrase structure rules:

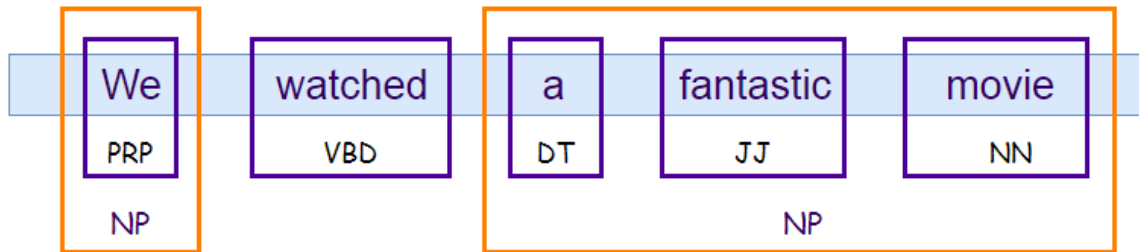
- $S(\text{Sentence}) \rightarrow NP VP$.
- $NP \rightarrow \{\text{Determiner, Noun, Pronoun, Proper name}\}$.
- $VP \rightarrow V (NP)(PP)(\text{Adverb})$.
- $PP \rightarrow \text{Pronoun (NP)}$.
- $AP \rightarrow \text{Adjective (PP)}$.



Example

Python Implementation:

- In the following example, we will extract a noun phrase from the text.
- Before extracting it, we need to define what kind of noun phrase we are looking for, or in other words, we have to **set the grammar** for a noun phrase.
- In this case, we **define a noun phrase by an optional determiner followed by adjectives and nouns**.
- Then we can define other rules to extract some other phrases.
- Next, we are going to use *RegexParser()* to parse the grammar.
- Notice that we can also visualize the text with the *.draw()* function.



```
#Extracting Noun Phrase from text :
```

```
# ? - optional character
```

```
# * - 0 or more repetitions
```

```
grammar = "NP : {<DT>?<JJ>*<NN>} "
```

```
#Creating a parser :
```

```
parser = nltk.RegexpParser(grammar)
```

```
#Parsing text :
```

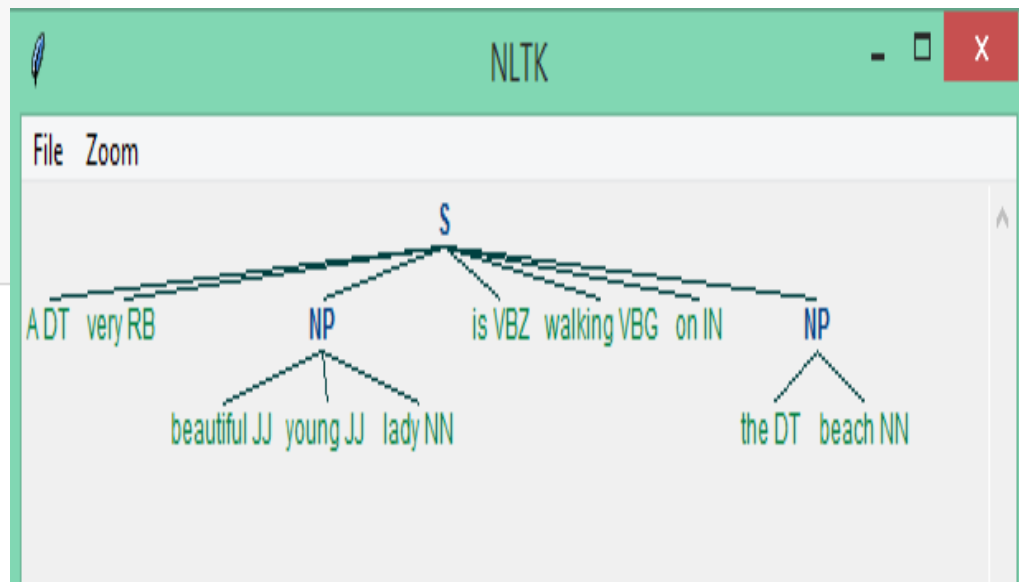
```
output = parser.parse(tagged_words)
```

```
print (output)
```

```
#To visualize :
```

```
output.draw()
```

```
(S
  A/DT
  very/RB
  (NP beautiful/JJ young/JJ lady/NN)
  is/VBZ
  walking/VBG
  on/IN
  (NP the/DT beach/NN))
```



Chinking:

- Chinking **excludes a part from our chunk.**
- There are certain situations where we need to exclude a part of the text from the whole text or chunk.
- In complex extractions, it is possible that chunking can output useless data.
- In such case scenarios, **we can use chinking to exclude some parts from that chunked text.**
- In the following example, we are going to take the whole string as a chunk, and then we are going to exclude adjectives from it by using chinking.
- We generally use chinking when we have a lot of useless data even after chunking.
- Hence, by using this method, we can easily set that apart,
- also to write chinking grammar, we have to use inverted curly braces, i.e.:
- **} write chinking grammar here {**

```
#Chinking example :  
# * - 0 or more repetitions  
# + - 1 or more repetitions
```

```
#Here we are taking the whole string and then  
#excluding adjectives from that chunk.
```

```
grammar = r""" NP: {<.*>+}  
               }<JJ>+{"""
```

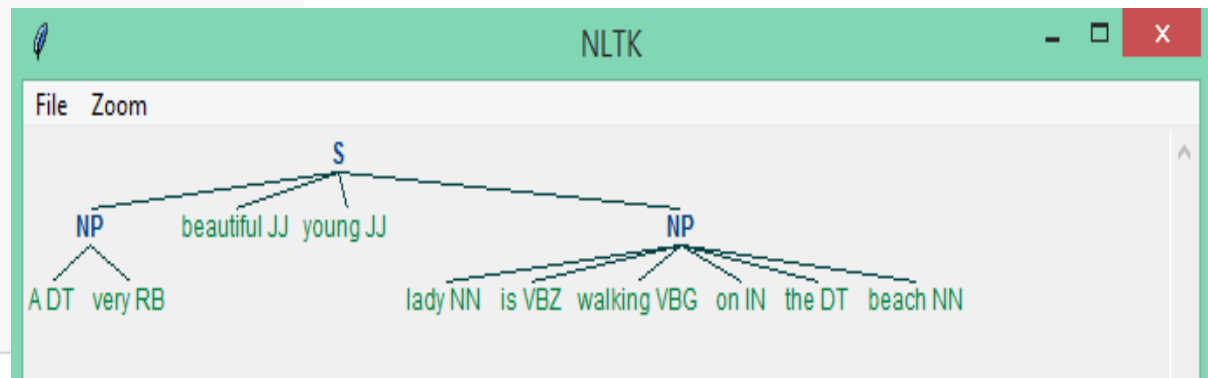
```
#Creating parser :  
parser = nltk.RegexpParser(grammar)
```

```
#parsing string :  
output = parser.parse(tagged_words)  
print(output)
```

```
#To visualize :  
output.draw()
```

```
(S  
  (NP A/DT very/RB)  
  beautiful/JJ  
  young/JJ  
  (NP lady/NN is/VBZ walking/VBG on/IN the/DT beach/NN))
```

- From the example above, we can see that adjectives separate from the other text.



Named Entity Recognition (NER):

- Named entity recognition can automatically scan entire articles **and pull out some fundamental entities** like people, organizations, places, date, time, money, and GPE discussed in them.
- **Use-Cases:**
 - Content classification for news channels.
 - Summarizing resumes.
 - Optimizing search engine algorithms.
 - Recommendation systems
 - Customer support

Commonly used types of named entity:

Named Entity Type	Example
ORGANIZATION	WHO
PERSON	President Obama
LOCATION	Mount Everest
DATE	2020-07-10
TIME	12:50 P.M.
MONEY	One Million Dollars
PERCENT	98.24%
FACILITY	Washington Monument
GPE	North West America

There are two options :

1. binary = True

When the binary value is True, then it will only show whether a particular entity is named entity or not. It will not show any further details on it.

2. binary = False

When the binary value equals False, it shows in detail the type of named entities.

```

#Sentence for NER :
sentence = "Mr. Smith made a deal on a beach of Switzerland near WHO."

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

#PoS tagging :
for w in tokenized_words:
    tagged_words = nltk.pos_tag(t_w)

#print (tagged_words)

#Named Entity Recognition :
N_E_R = nltk.ne_chunk(tagged_words,binary=True)

print(N_E_R)

#To visualize :
N_E_R.draw()

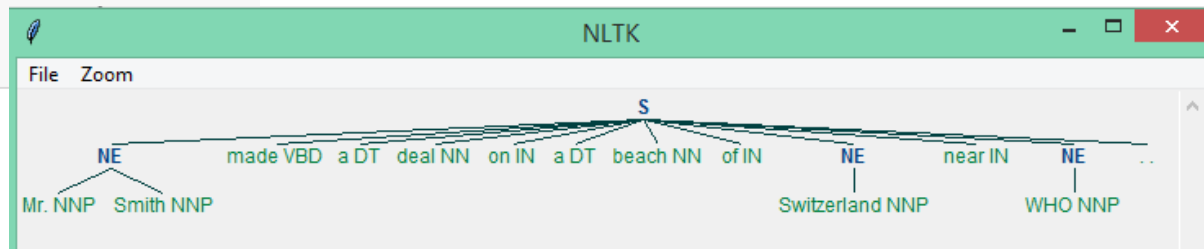
```

- Our graph does not show what type of named entity it is. It only shows whether a particular word is named entity or not.

```

(S
  (NE Mr./NNP Smith/NNP)
  made/VBD
  a/DT
  deal/NN
  on/IN
  a/DT
  beach/NN
  of/IN
  (NE Switzerland/NNP)
  near/IN
  (NE WHO/NNP)
  ./.)

```



Graph example of when a binary value is True.

```

#Sentence for NER :
sentence = "Mr. Smith made a deal on a beach of Switzerland near WHO."

#Tokenizing words :
tokenized_words = word_tokenize(sentence)

#PoS tagging :
for w in tokenized_words:
    tagged_words = nltk.pos_tag(t_w)

#print (tagged_words)

#Named Entity Recognition :
N_E_R = nltk.ne_chunk(tagged_words,binary=False)
print(N_E_R)

#To visualize :
N_E_R.draw()

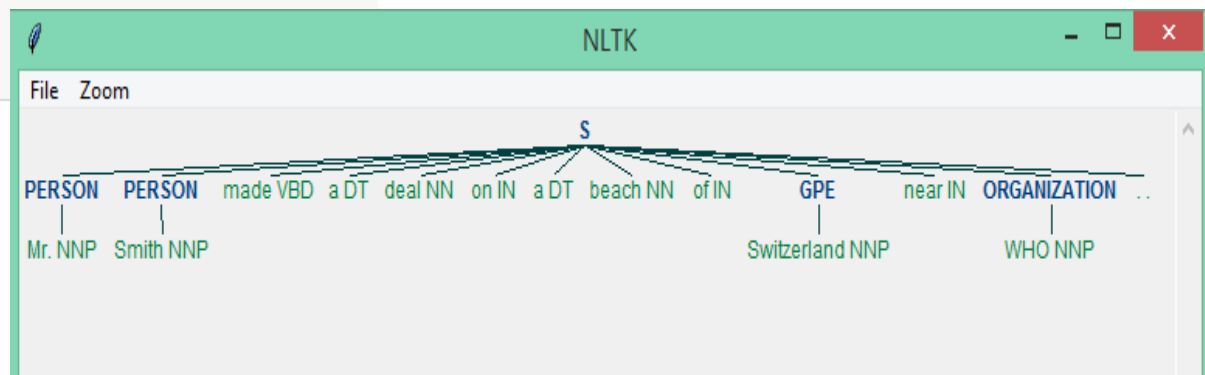
```

```

(S
 (PERSON Mr./NNP)
 (PERSON Smith/NNP)
 made/VBD
 a/DT
 deal/NN
 on/IN
 a/DT
 beach/NN
 of/IN
 (GPE Switzerland/NNP)
 near/IN
 (ORGANIZATION WHO/NNP)
 ./.)

```

- Our graph now shows what type of named entity it is.



Graph showing the type of named entities when a binary value equals false.

WordNet

- Wordnet is a **lexical database for the English language.**
- Wordnet **is a part of the NLTK corpus.**
- We can use Wordnet to:
 - find meanings of words,
 - synonyms,
 - antonyms, and
 - many other words.

- **We can check how many different definitions of a word are available in Wordnet.**

```
from nltk.corpus import wordnet  
  
for words in wordnet.synsets("Fun"):  
    print(words)
```

```
Synset('fun.n.01')  
Synset('fun.n.02')  
Synset('fun.n.03')  
Synset('playfulness.n.02')
```

- **We can also check the meaning of those different definitions.**

```
#How many differnt meanings :  
for words in wordnet.synsets("Fun"):  
    for lemma in words.lemmas():  
        print(lemma)  
    print("\n")
```

```
Lemma('fun.n.01.fun')  
Lemma('fun.n.01.merriment')  
Lemma('fun.n.01.playfulness')
```

```
Lemma('fun.n.02.fun')  
Lemma('fun.n.02.play')  
Lemma('fun.n.02.sport')
```

```
Lemma('fun.n.03.fun')
```

```
Lemma('playfulness.n.02.playfulness')  
Lemma('playfulness.n.02.fun')
```

Finding all the details of a specific word

```
word = wordnet.synsets("Play")[0]
```

```
#Checking name :
```

```
print(word.name())
```

```
#Checking definition :
```

```
print(word.definition())
```

```
#Checking examples:
```

```
print(word.examples())
```

```
play.n.01
```

```
a dramatic work intended for performance by actors on a stage
```

```
['he wrote several plays but only one was produced on Broadway']
```

All details for all meanings of a word.

```
#Word meaning with definitions :
for words in wordnet.synsets("Fun"):
    print(words.name())
    print(words.definition())
    print(words.examples())

    for lemma in words.lemmas():
        print(lemma)
    print("\n")
```

```
fun.n.01
activities that are enjoyable or amusing
['I do it for the fun of it', 'he is fun to have around']
Lemma('fun.n.01.fun')
Lemma('fun.n.01.merriment')
Lemma('fun.n.01.playfulness')
```

```
fun.n.02
verbal wit or mockery (often at another's expense but not to be taken seriously)
['he became a figure of fun', 'he said it in sport']
Lemma('fun.n.02.fun')
Lemma('fun.n.02.play')
Lemma('fun.n.02.sport')
```

```
fun.n.03
violent and excited activity
['she asked for money and then the fun began', 'they began to fight like fun']
Lemma('fun.n.03.fun')
```

```
playfulness.n.02
a disposition to find (or make) causes for amusement
['her playfulness surprised me', 'he was fun to be with']
Lemma('playfulness.n.02.playfulness')
Lemma('playfulness.n.02.fun')
```

Hypernyms

- **Hypernyms gives us a more abstract term for a word.**

```
word = wordnet.synsets("Play")[0]

#Find more abstract term :
print(word.hypernyms())

[Synset('dramatic_composition.n.01')]
```

Huponomms

- Hyponyms gives us a more specific term for a word.

```
word = wordnet.synsets("Play")[0]
```

```
#Find more specific term :  
word.hyponyms()
```

```
[Synset('grand_guignol.n.01'),  
 Synset('miracle_play.n.01'),  
 Synset('morality_play.n.01'),  
 Synset('mystery_play.n.01'),  
 Synset('passion_play.n.01'),  
 Synset('playlet.n.01'),  
 Synset('satyr_play.n.01'),  
 Synset('theater_of_the_absurd.n.01')]
```

Getting a name only

```
word = wordnet.synsets("Play")[0]  
  
#Get only name :  
print(word.lemmas()[0].name())
```

play

Synonyms

```
#Finding synonyms :  
  
#Empty list to store synonyms :  
synonyms = []  
  
for words in wordnet.synsets('Fun'):  
    for lemma in words.lemmas():  
        synonyms.append(lemma.name())  
  
synonyms
```

```
['fun',  
 'merriment',  
 'playfulness',  
 'fun',  
 'play',  
 'sport',  
 'fun',  
 'playfulness',  
 'fun']
```

Antonyms

```
#Finding antonyms :

#Empty list to store antonyms :
antonyms = []

for words in wordnet.synsets('Natural'):
    for lemma in words.lemmas():
        if lemma.antonyms():
            antonyms.append(lemma.antonyms()[0].name())

#Print antonyms :
antonyms

['unnatural', 'artificial', 'supernatural', 'sharp']
```

Synonyms and antonyms

```
#Finding synonyms and antonyms :

#Empty lists to store synonyms/antonyms :
synonyms = []
antonyms = []

for words in wordnet.synsets('New'):
    for lemma in words.lemmas():
        synonyms.append(lemma.name())
        if lemma.antonyms():
            antonyms.append(lemma.antonyms()[0].name())

#Print Lists :
print(synonyms)
print("\n")
print(antonyms)

['new', 'fresh', 'new', 'novel', 'raw', 'new', 'new', 'unexampled', 'new', 'new', 'newfangled', 'new', 'New', 'Modern', 'New',
'new', 'young', 'new', 'newly', 'freshly', 'fresh', 'new']

['old', 'worn']
```

Finding the similarity between words.

- Finding the similarity ratio between words using Wordnet.

```
#Similarity in words :  
word1 = wordnet.synsets("ship","n")[0]  
  
word2 = wordnet.synsets("boat","n")[0]  
  
#Check similarity :  
print(word1.wup_similarity(word2))  
  
0.9090909090909091
```

Similarity ratio between two words

- Using WordNet

```
#Similarity in words :  
word1 = wordnet.synsets("ship","n")[0]  
  
word2 = wordnet.synsets("boat","n")[0]  
  
#Check similarity :  
print(word1.wup_similarity(word2))
```

0.9090909090909091

```
#Similarity in words :  
word1 = wordnet.synsets("ship","n")[0]  
  
word2 = wordnet.synsets("bike","n")[0]  
  
#Check similarity :  
print(word1.wup_similarity(word2))
```

0.6956521739130435

Bag of Words:

- It is a **method of extracting essential features from raw text** so that we can use it for machine learning models.
- We call it “**Bag**” of words because we discard the order of occurrences of words.
- A bag of words model converts the raw text into words, and it also counts the frequency for the words in the text.
- In summary, a bag of words is a collection of words that represent a sentence along with the word count where the order of occurrences is not relevant.



Applications:

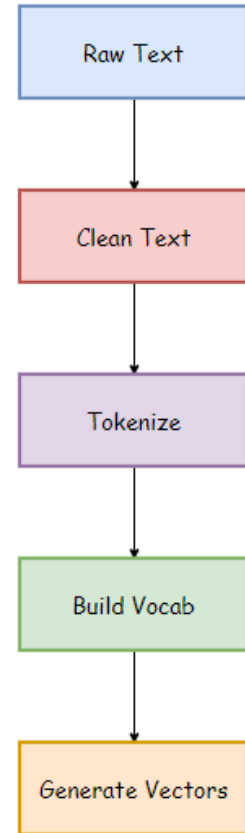
1. Natural language processing.
2. Information retrieval from documents.
3. Classifications of documents.

Limitations:

1. **Semantic meaning:** It does not consider the semantic meaning of a word. It ignores the context in which the word is used.
2. **Vector size:** For large documents, the vector size increase, which may result in higher computational time.
3. **Preprocessing:** In preprocessing, we need to perform data cleansing before using it.

Structure of a bag of words

- **Raw Text:** This is the original text on which we want analysis.
- **Clean Text:** Since our raw text contains some unnecessary punctuation marks and stopwords, so we need to clean it. Clean text is the text after removing such words.
- **Tokenize:** Tokenization represents the sentence as a list of words.
- **Building Vocab:** It contains total words used in the dataset, removing unnecessary data.
- **Generate Vocab:** It contains the words along with their indices in the sentences.



Example

- Sentences:
 1. Jim and Pam traveled by bus.
 2. The train was late.
 3. The flight was full. Traveling by flight is expensive.

a. Creating a basic structure:

1. Jim and Pam traveled by bus.
2. The train was late.
3. The flight was full. Traveling by flight is expensive.

Sentence 1	Sentence2	Sentence 3
Jim	The	The
and	train	flight
Pam	was	was
travelled	late	full
by		Travelling
the		by
bus		flight
		is
		expensive

b. Words with frequencies:

1. Jim and Pam traveled by bus.
2. The train was late.
3. The flight was full. Traveling by flight is expensive.

Sentence1	Count	Sentence2	Count	Sentence3	Count
Jim	1	The	1	The	1
and	1	train	1	flight	2
Pam	1	was	1	was	1
travelled	1	late	1	full	1
by	1			Travelling	1
the	1			by	1
bus	1			is	1
				expensive	1

c. Combining all the words:

Sentence	Frequency
and	1
bus	1
by	2
expensive	1
flight	2
full	1
is	1
jim	1
late	1
pam	1
the	3
train	1
travelled	1
travelling	1
was	1

- d. Final model:**

	and	bus	by	exp ensi ve	fligh t	full	is	jim	Late	pam	The	train	trav elle d	trav ellin g	was
S-1	1	1	1	0	0	0	0	1	0	1	1	0	1	0	0
S-2	0	0	0	0	0	0	0	0	1	0	1	1	0	0	1
S-3	0	0	1	1	2	1	1	0	0	0	1	0	0	1	1

Python Implementation

```
#Import required libraries :  
from sklearn.feature_extraction.text import CountVectorizer
```

```
#Text for analysis :  
sentences = ["Jim and Pam travelled by the bus:",  
             "The train was late",  
             "The flight was full.Travelling by flight is expensive"]
```

```
#Create an object :  
cv = CountVectorizer()
```

```
#Generating output for Bag of Words :  
B_O_W = cv.fit_transform(sentences).toarray()
```

```
#Total words with their index in model :
```

```
print(cv.vocabulary_)  
print("\n")
```

{'jim': 7, 'and': 0, 'pam': 9, 'travelled': 12, 'by': 2, 'the': 10, 'bus': 1, 'train': 11, 'was': 14, 'late': 8, 'flight': 4, 'full': 5, 'travelling': 13, 'is': 6, 'expensive': 3}

```
#Features :
```

```
print(cv.get_feature_names())  
print("\n")
```

['and', 'bus', 'by', 'expensive', 'flight', 'full', 'is', 'jim', 'late', 'pam', 'the', 'train', 'travelled', 'travelling', 'was']

```
[[1 1 1 0 0 0 0 1 0 1 1 0 1 0 0]  
 [0 0 0 0 0 0 0 0 1 0 1 1 0 0 1]  
 [0 0 1 1 2 1 1 0 0 0 1 0 0 1 1]]
```

```
#Show the output :  
print(B_O_W)
```

```
[[1 1 1 0 0 0 0 1 0 1 1 0 1 0 0]  
 [0 0 0 0 0 0 0 0 1 0 1 1 0 0 1]  
 [0 0 1 1 2 1 1 0 0 0 1 0 0 1 1]]
```

TF-IDF

- TF-IDF stands for **Term Frequency — Inverse Document Frequency**, which is a **scoring measure** generally used in information retrieval (IR) and summarization.
- The TF-IDF score **shows how important** or relevant a term is in a given document.

The intuition behind TF and IDF:

- If a **particular word appears multiple times** in a document, then it **might have higher importance** than the other words that appear fewer times (TF).
- At the same time, if a particular word appears many times in a document, but it is also present many times in some other documents, then maybe that word is frequent, so we cannot assign much importance to it (IDF).
- For instance, we have a database of thousands of dog descriptions, and the user wants to search for **“a cute dog”** from our database.
- The job of our search engine would be to display the closest response to the user query.
- **How would a search engine do that?**
- The search engine will possibly use TF-IDF to calculate the score for all of our descriptions, and the result with the higher score will be displayed as a response to the user.
- Now, this is the case when there is no exact match for the user's query.
- If there is an exact match for the user query, then that result will be displayed first.

- Then, let's suppose there are four descriptions available in our database.
 1. The furry dog.
 2. A cute doggo.
 3. A big dog.
 4. The lovely doggo.
- Notice that the first description contains 2 out of 3 words from our user query, and
- the second description contains 1 word from the query.
- The third description also contains 1 word, and
- the forth description contains no words from the user query.

- As we can sense that the closest answer to our query will be description number two, as it contains the essential word “**cute**” from the user’s query, this is how TF-IDF calculates the value.

- Notice that the term frequency values are the same for all of the sentences since none of the words in any sentences repeat in the same sentence.
- So, in this case, the value of TF will not be instrumental.
- Next, we are going to use IDF values to get the closest answer to the query.
- Notice that the word dog or doggo can appear in many many documents.
- Therefore, the IDF value is going to be very low.
- Eventually, the TF-IDF value will also be lower.
- However, if we check the word “cute” in the dog descriptions, then it will come up relatively fewer times, so it increases the TF-IDF value.
- So the word “cute” has more discriminative power than “dog” or “doggo.”
- Then, our search engine will find the descriptions that have the word “cute” in it, and in the end, that is what the user was looking for.
- Simply put, the higher the $TF \cdot IDF$ score, the rarer or unique or valuable the term and vice versa.

Example:

- **Sentence 1:** This is the first document.
- **Sentence 2:** This document is the second document.

TF: Term Frequency

$$TF = \frac{\textit{Frequency of the word in the sentence}}{\textit{Total number of words in the sentence}}$$

a. Represent the words of the sentences in the table.

Sentence 1	Sentence 2
This	This
is	document
the	is
first	the
document	second
	document

b. Displaying the frequency of words.

Sentence 1	TF	Sentence 2	TF
This	1	This	1
is	1	document	2
the	1	is	1
first	1	the	1
document	1	second	1

c. Calculating TF using a formula.

$$TF = \frac{\text{Frequency of the word in the sentence}}{\text{Total number of words in the sentence}}$$

Sentence 1	TF	Sentence 2	TF
This	$\frac{1}{5} = 0.20$	This	$\frac{1}{6} = 0.166$
is	$\frac{1}{5} = 0.20$	document	$\frac{1}{3} = 0.33$
the	$\frac{1}{5} = 0.20$	is	$\frac{1}{6} = 0.166$
first	$\frac{1}{5} = 0.20$	the	$\frac{1}{6} = 0.166$
document	$\frac{1}{5} = 0.20$	second	$\frac{1}{6} = 0.166$

- **IDF: Inverse Document Frequency**

$$IDF = \frac{\text{Total number of sentences}}{\text{Number of sentences containing that word}}$$

Sentence 1	IDF	Sentence 2	IDF
This	$2/2 = 1$	This	$2/2 = 1$
is	$2/2 = 1$	document	$2/2 = 1$
the	$2/2 = 1$	is	$2/2 = 1$
first	$2/1 = 2$	the	$2/2 = 1$
document	$2/2 = 1$	second	$2/1 = 2$

e. Calculating TF-IDF.

TF-IDF is the multiplication of TF*IDF

Word	TF-IDF 1	TF-IDF2
This	$0.20 * 1 = 0.20$	$0.166 * 1 = 0.16$
is	$0.20 * 1 = 0.20$	$0.166 * 1 = 0.16$
the	$0.20 * 1 = 0.20$	$0.166 * 1 = 0.16$
first	$0.20 * 2 = 0.40$	$0 * 2 = 0$
document	$0.20 * 1 = 0.20$	$0.333 * 1 = 0.33$
second	$0 * 2 = 0$	$0.166 * 2 = 0.33$

- In this case, notice that the important words that discriminate both the sentences are “first” in sentence-1 and “second” in sentence-2 as we can see, those words have a relatively higher value than other words.
- However, there are many variations for smoothing out the values for large documents. The most common variation is to use a log value for TF-IDF. Let’s calculate the TF-IDF value again by using the new IDF value.

$$IDF = \log \left(\frac{\text{Total number of sentences}}{\text{Number of sentences containing that word}} \right)$$

f. Calculating IDF value using log.

Word	IDF 1	IDF 2
This	$\log(1) = 0$	$\log(1) = 0$
is	$\log(1) = 0$	$\log(1) = 0$
the	$\log(1) = 0$	$\log(1) = 0$
first	$\log(2) = 0.693$	$\log(2) = 0.693$
document	$\log(1) = 0$	$\log(1) = 0$
second	$\log(2) = 0.693$	$\log(2) = 0.693$

g. Calculating TF-IDF.

As seen above, “first” and “second” values are important words that help us to distinguish between those two sentences.

- Now that we saw the basics of TF-IDF. Next, we are going to use the sklearn library to implement TF-IDF in Python. A different formula calculates the actual output from our program. First, we will see an overview of our calculations and formulas, and then we will implement it in Python.

Word	TF-IDF 1	TF-IDF 2
This	$0.20 * \log(1) = 0$	$0.166 * \log(1) = 0$
is	$0.20 * \log(1) = 0$	$0.333 * \log(1) = 0$
the	$0.20 * \log(1) = 0$	$0.166 * \log(1) = 0$
first	$0.20 * \log(2) = 0.13$	$0 * \log(2) = 0$
document	$0.20 * \log(1) = 0$	$0.166 * \log(1) = 0$
second	$0 * \log(2) = 0$	$0.166 * \log(2) = 0.13$

Actual Calculations:

a. Term Frequency (TF):

Sentence 1	TF	Sentence 2	TF
This	1	This	1
is	1	document	2
the	1	is	1
first	1	the	1
document	1	second	1

b. Inverse Document Frequency (IDF):

$$IDF = \log \left(\frac{\text{Total number of sentences} + 1}{\text{Number of sentences containing that word} + 1} \right)$$

Word	IDF 1	IDF 2
This	$\log(3/3) = 0$	$\log(3/3) = 0$
is	$\log(3/3) = 0$	$\log(3/3) = 0$
the	$\log(3/3) = 0$	$\log(3/3) = 0$
first	$\log(3/2) = 0.40$	$\log(3/2) = 0.40$
document	$\log(3/3) = 0$	$\log(3/3) = 0$
second	$\log(3/2) = 0.40$	$\log(3/2) = 0.40$

c. Calculate TF-IDF values:

$$TF - IDF = (TF) * (IDF + 1)$$

Word	TF-IDF 1	TF-IDF 2
This	$1*(0+1) = 1$	$1*(0+1) = 1$
is	$1*(0+1) = 1$	$1*(0+1) = 1$
the	$1*(0+1) = 1$	$1*(0+1) = 1$
first	$1*(0.40+1) = 1.40$	$0*(0.40+1) = 0$
document	$1*(0+1) = 1$	$2*(0+1) = 2$
second	$0*(0.40+1) = 0$	$1*(0.40+1) = 1.40$

Python Implementation:

```
#Import required libraries :
from sklearn.feature_extraction.text import TfidfVectorizer

#Sentences for analysis :
sentences = ['This is the first document','This document is the second document']

#Create an object :
vectorizer = TfidfVectorizer(norm = None)

#Generating output for TF_IDF :
X = vectorizer.fit_transform(sentences).toarray()

#Total words with their index in model :
print(vectorizer.vocabulary_)
print("\n")

#Features :
print(vectorizer.get_feature_names())
print("\n")

#Show the output :
print(X)
```

{'this': 5, 'is': 2, 'the': 4, 'first': 1, 'document': 0, 'second': 3}

['document', 'first', 'is', 'second', 'the', 'this']

[1.	1.40546511	1.	0.	1.	1.	]
[2.	0.	1.	1.40546511	1.	1.	]]

References:

- <https://towardsai.net/p/nlp/natural-language-processing-nlp-with-python-tutorial-for-beginners-1f54e610a1a0>

End of chapter 1